

Tutoring Marketplace

Codebase report: business logic, architecture and a worked example

Course: Corporate Development (CSS-3008) · Lab 2: Join Spring Boot to your product
Java 21 · Spring Boot 3.5 · Maven

Contents

- 1. What the product does
- 2. Business logic: statuses and rules
- 3. Architecture: the five packages
- 4. Every file explained, with its code
- 5. How Spring puts it together at start-up
- 6. Worked example: one lesson from start to end
- 7. Tests
- 8. How to run it
- 9. Lab 2 requirements: where each one is met
- 10. Quick map: where each piece of logic lives

1. What the product does

The project is the back end of a **tutoring marketplace**. A student finds a tutor, books a time slot and pays through the platform. The platform does not pass the money on straight away: it **holds** the payment until the lesson has taken place, then pays the tutor and keeps a **commission**.

This means money moves every time a lesson changes status. Taking the payment, paying the tutor and refunding the student are each triggered by a status change. If a lesson could jump to the wrong status, the platform could pay a tutor for a lesson nobody paid for, or refund money it no longer has. That is why the allowed order of statuses is enforced in code instead of being left to whoever edits the record.

The core item of the system is a **lesson**: one student, one tutor, one time slot. The code answers one question over and over: *"is this lesson allowed to move from status A to status B?"*

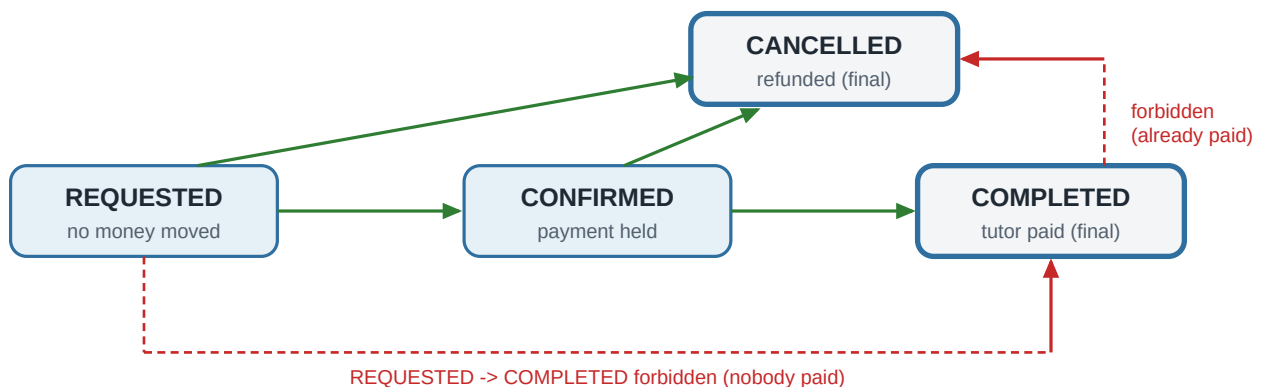
2. Business logic: statuses and rules

2.1 The four statuses

A lesson always has exactly one status. They are defined in **LessonStatus**:

Status	What it means	Money
REQUESTED	The student asked the tutor for a slot.	Nothing has moved yet.
CONFIRMED	The tutor accepted the slot.	The student's payment is taken and held by the platform.
COMPLETED	The lesson took place. Final .	The tutor is paid; the platform keeps its commission.
CANCELLED	Called off before it took place. Final .	Any held payment goes back to the student.

2.2 Which moves are allowed



Green arrows are allowed moves; red dashed arrows are the two forbidden moves the business cares about most.

From	To	Result	Why
REQUESTED	CONFIRMED	Allowed	The tutor accepted, so the payment is taken and held.
REQUESTED	CANCELLED	Allowed	Nobody paid yet, so nothing has to be refunded.
CONFIRMED	COMPLETED	Allowed	The lesson happened, so the held payment goes to the tutor minus commission.
CONFIRMED	CANCELLED	Allowed	Called off in time, so the held payment goes back to the student.
REQUESTED	COMPLETED	Forbidden	Nobody paid. The platform would pay the tutor out of its own pocket and report commission that never arrived.
COMPLETED	CANCELLED	Forbidden	The tutor is already paid. A refund would use money the platform no longer holds.
anything else		Forbidden	COMPLETED and CANCELLED are final, and a lesson never moves backwards.

2.3 Two kinds of rule

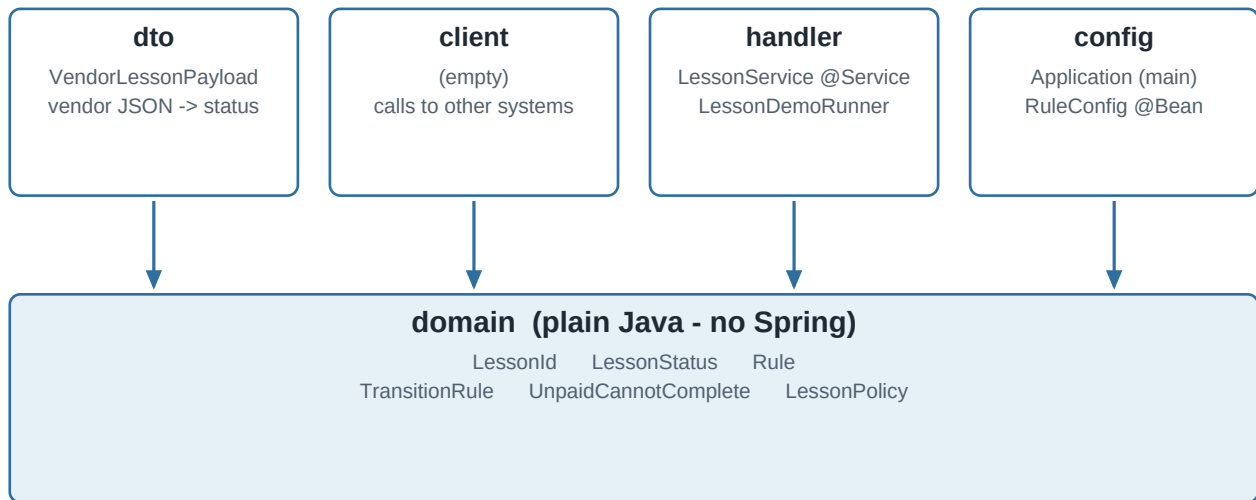
The lab asks for the business logic to be split into **separate rules** behind one interface, instead of one long *if* block. This project has two:

- **TransitionRule: the status table.** It knows every allowed move and rejects everything else.
- **UnpaidCannotComplete: the stop-factor.** It knows one business fact: the platform only holds money from CONFIRMED on, so a lesson that was never paid for cannot be completed. It ignores every other move.

Note. Both rules forbid REQUESTED -> COMPLETED. That is on purpose: the stop-factor runs first, so the user gets the more useful message *"An unpaid lesson cannot be completed"* instead of the generic *"Cannot move a lesson from REQUESTED to COMPLETED"*. If someone changed the table later, the payment rule would still protect the money.

3. Architecture: the five packages

All code is in `src/main/java/com/endev/tutoring/`, split into five packages. The course calls them *rings*: **domain** is in the centre, and the four outer packages sit around it.



Arrows point inward: the outer packages use domain; domain uses none of them.

The one rule of this layout: dependencies point **inward only**. Outer packages may import **domain**, but **domain** never imports an outer package and never imports Spring (`org.springframework`). You can check it yourself:

```
grep -rn springframework src/main/java/com/endev/tutoring/domain
# no output = no Spring in domain
```

Package	Job	Uses Spring?	Files
domain	The business itself: statuses and rules. Decides whether a move is allowed.	No, never	LessonId, LessonStatus, Rule, TransitionRule, UnpaidCannotComplete, LessonPolicy
config	Starts Spring and wires the objects together.	Yes	Application, RuleConfig
handler	The way in : receives a request and passes it to the domain.	Yes	LessonService, LessonDemoRunner
dto	Translates data from outside (a vendor's JSON) into the app's own types.	No	VendorLessonPayload
client	The way out : calls to other systems. Empty in this lab.	-	package-info.java only

Why bother?

- **The business rules can't be broken by framework changes.** Upgrading Spring, adding HTTP in week 9 or adding a database later only touches the outer packages.
- **The rules are easy to test.** A domain test is just `new TransitionRule().check(...)`; no Spring context has to start.
- **You always know where to look.** A wrong status decision is in domain; a wiring problem is in config; a wrong vendor mapping is in dto.

4. Every file explained, with its code

Line numbers below are the real line numbers in the files, so you can jump straight to them in your editor.

4.1 domain: the rules

LessonStatus is an enum, so a status can only ever be one of these four values. A typo like "CONFIRMED" does not compile.

```
1 package com.endev.tutoring.domain;
2
3 public enum LessonStatus {
4     REQUESTED,
5     CONFIRMED,
6     COMPLETED,
7     CANCELLED
8 }
```

src/main/java/com/endev/tutoring/domain/LessonStatus.java

LessonId is a record (an immutable value). Its constructor (lines 5-8) refuses a null or blank ID, so a LessonId that exists is always valid.

```
1 package com.endev.tutoring.domain;
2
3 public record LessonId(String value) {
4
5     public LessonId {
6         if (value == null || value.isBlank()) {
7             throw new IllegalArgumentException("Lesson id must not be empty");
8         }
9     }
10 }
```

src/main/java/com/endev/tutoring/domain/LessonId.java

Rule is the interface every rule implements. It has one method. A rule returns normally when the move is fine and throws **IllegalStateException** when it isn't. This is what lets Spring inject "a Rule" without knowing which rules are inside.

```
1 package com.endev.tutoring.domain;
2
3 public interface Rule {
4
5     // throws IllegalStateException if the move is not allowed
6     void check(LessonStatus from, LessonStatus to);
7 }
```

src/main/java/com/endev/tutoring/domain/Rule.java

TransitionRule is the status table from section 2.2 written as code. It starts from **allowed = false** (line 7), then opens up only the moves in the table: REQUESTED may go to CONFIRMED or CANCELLED (lines 9-10); CONFIRMED may go to COMPLETED or CANCELLED (lines 11-12). COMPLETED and CANCELLED have no branch, so from them nothing is allowed. That is how "final" is enforced. Anything not allowed throws at line 17.

```

1 package com.endev.tutoring.domain;
2
3 public class TransitionRule implements Rule {
4
5     @Override
6     public void check(LessonStatus from, LessonStatus to) {
7         boolean allowed = false;
8
9         if (from == LessonStatus.REQUESTED) {
10             allowed = to == LessonStatus.CONFIRMED || to == LessonStatus.CANCELLED;
11         } else if (from == LessonStatus.CONFIRMED) {
12             allowed = to == LessonStatus.COMPLETED || to == LessonStatus.CANCELLED;
13         }
14         // COMPLETED and CANCELLED are final, nothing can come after them
15
16         if (!allowed) {
17             throw new IllegalStateException("Cannot move a lesson from " + from + " to " + to);
18         }
19     }
20 }

```

src/main/java/com/endev/tutoring/domain/TransitionRule.java

UnpaidCannotComplete is the stop-factor. Line 8 checks the single case it cares about; every other move passes straight through, because the order of statuses is not its job.

```

1 package com.endev.tutoring.domain;
2
3 public class UnpaidCannotComplete implements Rule {
4
5     @Override
6     public void check(LessonStatus from, LessonStatus to) {
7         // the student only pays when the lesson is CONFIRMED
8         if (from == LessonStatus.REQUESTED && to == LessonStatus.COMPLETED) {
9             throw new IllegalStateException("An unpaid lesson cannot be completed");
10        }
11    }
12 }

```

src/main/java/com/endev/tutoring/domain/UnpaidCannotComplete.java

LessonPolicy glues the rules together. It receives a list of rules (line 9) and, in **check**, first rejects a missing status (lines 15-17), then runs each rule in order (lines 18-20). The first rule that throws stops the move. Because LessonPolicy itself *implements Rule*, the whole chain looks like a single Rule to the outside. **move** (lines 23-26) is a convenience: check, then return the new status.

```

1 package com.endev.tutoring.domain;
2
3 import java.util.List;
4
5 public class LessonPolicy implements Rule {
6     private final List<Rule> rules;
7
8     public LessonPolicy(List<Rule> rules) {
9         this.rules = rules;
10    }
11
12    @Override
13    public void check(LessonStatus from, LessonStatus to) {
14        if (from == null || to == null) {
15            throw new IllegalArgumentException("Status must not be null");
16        }
17        for (Rule rule : rules) {
18            rule.check(from, to);
19        }
20    }
21
22    public LessonStatus move(LessonStatus from, LessonStatus to) {
23        check(from, to);
24        return to;
25    }
26 }
27 }

```

src/main/java/com/endev/tutoring/domain/LessonPolicy.java

4.2 config: Spring set-up

Application is where the program starts. **SpringApplication.run** (line 11) boots Spring. Line 7 matters: Spring normally only scans the package the class sits in (**config**), so **scanBasePackages** tells it to scan the whole project and find the service in **handler** too.

```

1 package com.endev.tutoring.config;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 // Application is in the config package, so we tell Spring to scan the whole project
7 @SpringBootApplication(scanBasePackages = "com.endev.tutoring")
8 public class Application {
9
10    public static void main(String[] args) {
11        SpringApplication.run(Application.class, args);
12    }
13 }

```

src/main/java/com/endev/tutoring/config/Application.java

RuleConfig is the only place Spring touches the rules. The **@Bean** method (lines 15-19) builds the chain once at start-up. **Line 17 decides the order**: the stop-factor first, then the status table. Spring keeps the returned object and hands it to anyone who asks for a **Rule**. This is why the rule classes need no Spring annotations: Spring meets them here, in config.

```

1 package com.endev.tutoring.config;
2
3 import com.endev.tutoring.domain.LessonPolicy;
4 import com.endev.tutoring.domain.Rule;
5 import com.endev.tutoring.domain.TransitionRule;
6 import com.endev.tutoring.domain.UnpaidCannotComplete;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 import java.util.List;
11
12 @Configuration
13 public class RuleConfig {
14
15     @Bean
16     public Rule lessonRules() {
17         List<Rule> rules = List.of(new UnpaidCannotComplete(), new TransitionRule());
18         return new LessonPolicy(rules);
19     }
20 }

```

src/main/java/com/endev/tutoring/config/RuleConfig.java

4.3 handler: the way in

LessonService is the **@Service** the lab asks for. Its constructor (line 12) takes a **Rule**. Spring sees that and passes in the bean from RuleConfig. This is **dependency injection**: the service never creates its rules itself, so you could give it different rules in a test without changing it. **move** (lines 16-19) owns no logic: it asks the rules and returns the new status.

```

1 package com.endev.tutoring.handler;
2
3 import com.endev.tutoring.domain.LessonStatus;
4 import com.endev.tutoring.domain.Rule;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 public class LessonService {
9
10     private final Rule rules;
11
12     public LessonService(Rule rules) {
13         this.rules = rules;
14     }
15
16     public LessonStatus move(LessonStatus from, LessonStatus to) {
17         rules.check(from, to);
18         return to;
19     }
20 }

```

src/main/java/com/endev/tutoring/handler/LessonService.java

LessonDemoRunner is a **CommandLineRunner**: Spring runs it once, right after start-up. It exists so that *mvn spring-boot:run* shows the rules working. It turns a vendor payload into a lesson (lines 21-24), tries four moves (lines 26-29), and for each one prints "allowed" or "forbidden" (lines 34-43). On a forbidden move it keeps the old status (line 41), just like the real system would.

```

1 package com.endev.tutoring.handler;
2
3 import com.endev.tutoring.domain.LessonId;
4 import com.endev.tutoring.domain.LessonStatus;
5 import com.endev.tutoring.dto.VendorLessonPayload;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 // Runs when the app starts and shows which moves are allowed
10 @Component
11 public class LessonDemoRunner implements CommandLineRunner {
12
13     private final LessonService lessonService;
14
15     public LessonDemoRunner(LessonService lessonService) {
16         this.lessonService = lessonService;
17     }
18
19     @Override
20     public void run(String... args) {
21         VendorLessonPayload payload = new VendorLessonPayload("LES-2026-0042", "PENDING");
22         LessonId id = payload.toLessonId();
23         LessonStatus status = payload.toLessonStatus();
24         System.out.println("Lesson " + id.value() + " arrived from the vendor as " + status);
25
26         status = tryMove(status, LessonStatus.COMPLETED);
27         status = tryMove(status, LessonStatus.CONFIRMED);
28         status = tryMove(status, LessonStatus.COMPLETED);
29         status = tryMove(status, LessonStatus.CANCELLED);
30
31         System.out.println("Lesson " + id.value() + " ends as " + status);
32     }
33
34     private LessonStatus tryMove(LessonStatus from, LessonStatus to) {
35         try {
36             lessonService.move(from, to);
37             System.out.println(" " + from + " -> " + to + ": allowed");
38             return to;
39         } catch (IllegalStateException e) {
40             System.out.println(" " + from + " -> " + to + ": forbidden (" + e.getMessage() + ")");
41             return from;
42         }
43     }
44 }

```

src/main/java/com/endev/tutoring/handler/LessonDemoRunner.java

4.4 dto: translating outside data

VendorLessonPayload models the JSON a scheduling vendor sends. The vendor uses its own words (PENDING, BOOKED, FINISHED, CANCELED) and this class translates them into LessonStatus (lines 17-25). Unknown or missing values are rejected (lines 14-16 and 26-27). This keeps the vendor's vocabulary out of the domain: if the vendor renames a status, only this file changes.

```

1 package com.endev.tutoring.dto;
2
3 import com.endev.tutoring.domain.LessonId;
4 import com.endev.tutoring.domain.LessonStatus;
5
6 // JSON from the vendor looks like: {"lessonId": "LES-2026-0042", "status": "BOOKED"}
7 public record VendorLessonPayload(String lessonId, String status) {
8
9     public LessonId toLessonId() {
10         return new LessonId(lessonId);
11     }
12
13     public LessonStatus toLessonStatus() {
14         if (status == null) {
15             throw new IllegalArgumentException("Vendor status is null");
16         }
17         switch (status) {
18             case "PENDING":
19                 return LessonStatus.REQUESTED;
20             case "BOOKED":
21                 return LessonStatus.CONFIRMED;
22             case "FINISHED":
23                 return LessonStatus.COMPLETED;
24             case "CANCELED":
25                 return LessonStatus.CANCELLED;
26             default:
27                 throw new IllegalArgumentException("Unknown vendor lesson status: " + status);
28         }
29     }
30 }

```

src/main/java/com/endev/tutoring/dto/VendorLessonPayload.java

4.5 client

Empty in this lab. It only contains **package-info.java** because git does not keep empty folders. Later, outgoing calls (a payment provider, a database) go here.

5. How Spring puts it together at start-up

When you run `mvn spring-boot:run`, this happens before any lesson is touched:

Step	What happens	Where
1	<code>main()</code> calls <code>SpringApplication.run()</code> .	Application.java:11
2	Spring scans every package under <code>com.endev.tutoring</code> for annotated classes: it finds <code>@Configuration RuleConfig</code> , <code>@Service LessonService</code> and <code>@Component LessonDemoRunner</code> .	Application.java:7
3	Spring calls <code>lessonRules()</code> : it creates <code>UnpaidCannotComplete</code> and <code>TransitionRule</code> , wraps them in a <code>LessonPolicy</code> , and stores it as the Rule bean.	RuleConfig.java:15-19
4	Spring creates <code>LessonService</code> . Its constructor needs a Rule, so Spring passes the <code>LessonPolicy</code> from step 3.	LessonService.java:12
5	Spring creates <code>LessonDemoRunner</code> . Its constructor needs a <code>LessonService</code> , so Spring passes the one from step 4.	LessonDemoRunner.java:15
6	The app has started. Spring calls <code>LessonDemoRunner.run()</code> , which walks one lesson through the rules.	LessonDemoRunner.java:20
7	There is no web server in this lab, so once <code>run()</code> returns, the program exits.	-

Key idea: nobody calls new LessonService(...). You describe *what* each class needs (through its constructor) and Spring decides *how* to build and connect them. That is the whole point of "joining Spring Boot" in this lab.

6. Worked example: one lesson from start to end

This is exactly what the demo runner does when the app starts. We follow lesson **LES-2026-0042** step by step and note which line of code makes each decision.

Step 1: the vendor sends the lesson

The scheduling vendor reports a new booking request:

```
{ "lessonId": "LES-2026-0042", "status": "PENDING" }
```

`VendorLessonPayload.toLessonId()` turns the ID into a `LessonId` (the constructor checks it isn't blank). `toLessonStatus()` hits `case "PENDING"` and returns `REQUESTED` (`VendorLessonPayload.java:18-19`).

```
Lesson LES-2026-0042 arrived from the vendor as REQUESTED
```

Step 2: someone tries to complete it without paying: REQUESTED -> COMPLETED

Imagine a tutor trying to mark the lesson done so they get paid. The call goes:

```
LessonDemoRunner.tryMove          (LessonDemoRunner.java:36)
-> LessonService.move             (LessonService.java:17) rules.check(REQUESTED, COMPLETED)
-> LessonPolicy.check             (LessonPolicy.java:15)  null check ..... passes
  -> UnpaidCannotComplete.check (UnpaidCannotComplete.java:8)
    from == REQUESTED && to == COMPLETED -> TRUE -> throw
    (TransitionRule never runs: the first failing rule stops the chain)
  <- IllegalStateException("An unpaid lesson cannot be completed")
LessonDemoRunner catches it (line 39) and keeps the old status REQUESTED (line 41)
```

```
REQUESTED -> COMPLETED: forbidden (An unpaid lesson cannot be completed)
```

Business meaning: no payment was taken, so the platform refuses to pay the tutor. No money is lost.

Step 3: the tutor accepts: REQUESTED -> CONFIRMED

```
LessonPolicy.check
  null check ..... passes
  UnpaidCannotComplete: not (REQUESTED -> COMPLETED) ..... passes
  TransitionRule: from == REQUESTED (line 9)
    allowed = (to == CONFIRMED || ...) ..... TRUE -> passes
LessonService.move returns CONFIRMED
```

```
REQUESTED -> CONFIRMED: allowed
```

Business meaning: the platform now takes the student's payment and holds it.

Step 4: the lesson happens: CONFIRMED -> COMPLETED

```
UnpaidCannotComplete: from is CONFIRMED, not REQUESTED .. passes
TransitionRule: from == CONFIRMED (line 11)
  allowed = (to == COMPLETED || ...) ..... TRUE -> passes
LessonService.move returns COMPLETED
```

```
CONFIRMED -> COMPLETED: allowed
```

Business meaning: the held payment goes to the tutor, minus the platform's commission.

Step 5: the student asks to cancel after the fact: COMPLETED -> CANCELLED

```
UnpaidCannotComplete: not its case ..... passes
TransitionRule: from == COMPLETED -> no branch matches
  allowed stays false (line 7) -> throw (line 17)
<- IllegalStateException("Cannot move a lesson from COMPLETED to CANCELLED")
status stays COMPLETED
```

```
COMPLETED -> CANCELLED: forbidden (Cannot move a lesson from COMPLETED to CANCELLED)
Lesson LES-2026-0042 ends as COMPLETED
```

Business meaning: the tutor has already been paid, so a cancellation would refund money the platform no longer holds. A dispute about a finished lesson must be handled as a separate refund case.

6.1 Two more cases the code handles

- **A missing status.** `lessonService.move(null, CONFIRMED)` is stopped at `LessonPolicy.java:15` with `IllegalArgumentException("Status must not be null")` before any rule runs.
- **A vendor status we don't know.** `new VendorLessonPayload("LES-1", "REFUNDED").toLessonStatus()` hits `default` at `VendorLessonPayload.java:26` and throws, so an unknown value never reaches the domain.

6.2 Example: adding a new rule later

Say the business decides a lesson can't be confirmed if the tutor is blocked. You would:

- write one new class in `domain`, e.g. `BlockedTutorCannotConfirm` implements `Rule`;
- add it to the list in `RuleConfig.java:17`;
- write a small test for it.

Nothing else changes: not `LessonService`, not the other rules. That is the benefit of the `Rule` interface.

7. Tests

Tests are in `src/test/java/com/endev/tutoring/` and mirror the main packages. They run with `mvn -q verify`: 35 tests, all green.

Test class	What it proves	Tests
<code>TransitionRuleTest</code>	Each allowed move passes; forbidden moves (skip payment, after final, going back) throw with the right message.	8
<code>UnpaidCannotCompleteTest</code>	The unpaid case throws; a paid lesson can be completed; other moves are ignored.	3
<code>LessonPolicyTest</code>	The README status table through the whole chain, the rule order, and the null check.	7
<code>LessonIdTest</code>	An ID keeps its value; null, empty and blank IDs are rejected.	4
<code>VendorLessonPayloadTest</code>	Every vendor status maps to the right <code>LessonStatus</code> ; unknown, lowercase and null are rejected.	8
<code>LessonServiceTest</code>	With real Spring: the injected chain runs both rules and rejects null.	4
<code>ApplicationTest</code>	The Spring context starts and contains the <code>Rule</code> bean and the service.	1

The domain tests create objects with `new` and need no Spring; they run in milliseconds. Only the last two use `@SpringBootTest`, because their job is to check the wiring.

An example test, one case per method:

```
11     private final TransitionRule rule = new TransitionRule();
12
13     @Test
14     void requestedToConfirmedIsAllowed() {
15         assertDoesNotThrow(() -> rule.check(LessonStatus.REQUESTED, LessonStatus.CONFIRMED));
16     }
17
18     @Test
19     void requestedToCancelledIsAllowed() {
20         assertDoesNotThrow(() -> rule.check(LessonStatus.REQUESTED, LessonStatus.CANCELLED));
21     }
```

src/test/java/com/endev/tutoring/domain/TransitionRuleTest.java (lines 11-21)

8. How to run it

You need Java 21 or newer and Maven. From the project folder:

```
mvn -q verify          # compile and run all tests; no output means everything passed
mvn spring-boot:run    # start the app
```

Expected output of *mvn spring-boot:run* (after the Spring banner):

```
Started Application in 0.5 seconds
Lesson LES-2026-0042 arrived from the vendor as REQUESTED
  REQUESTED -> COMPLETED: forbidden (An unpaid lesson cannot be completed)
  REQUESTED -> CONFIRMED: allowed
  CONFIRMED -> COMPLETED: allowed
  COMPLETED -> CANCELLED: forbidden (Cannot move a lesson from COMPLETED to CANCELLED)
Lesson LES-2026-0042 ends as COMPLETED
```

The program then exits on its own: this lab has no web server, so there is nothing to keep it running.

9. Lab 2 requirements: where each one is met

Requirement	Where it is met
Same product as Lab 1	Tutoring marketplace: LessonId, LessonStatus, LessonPolicy. No Ticket* names.
Spring Boot parent POM, Application in config	pom.xml (spring-boot-starter-parent 3.5.16); config/Application.java
A @Service injects Rule	handler/LessonService.java:7-14
Two types implement Rule: status table + one stop-factor	domain/TransitionRule.java and domain/UnpaidCannotComplete.java
Rule chain wired with a @Bean in config	config/RuleConfig.java:15-19
Policy and rules have no org.springframework import	Everything in domain/; the grep in section 3 returns nothing
Packages dto / domain / client / handler / config	All five exist; client is empty by design
dto maps a vendor JSON shape onto the status	dto/VendorLessonPayload.java:17-27
README with the package diagram	README.md, section "Package diagram"
No REST, no database, no Docker	pom.xml has only spring-boot-starter and the test starter
mvn -q verify is green	35 tests, 0 failures
mvn spring-boot:run starts	See section 8
No secrets, .env or target/ in git	.gitignore covers target/; git history contains no secret files

10. Quick map: where each piece of logic lives

Logic	File : lines
The four statuses	domain/LessonStatus.java : 3-8
Lesson ID can't be empty	domain/LessonId.java : 5-8
Rule contract	domain/Rule.java : 6
Status table: from REQUESTED	domain/TransitionRule.java : 9-10
Status table: from CONFIRMED	domain/TransitionRule.java : 11-12
Status table: finals / reject	domain/TransitionRule.java : 7, 16-18
Stop-factor: unpaid can't complete	domain/UnpaidCannotComplete.java : 8-10
Null status rejected	domain/LessonPolicy.java : 15-17
Run rules in order	domain/LessonPolicy.java : 18-20
Program entry point	config/Application.java : 10-12
Rule order is decided	config/RuleConfig.java : 17
Rule bean for Spring	config/RuleConfig.java : 15-19
Service gets Rule injected	handler/LessonService.java : 12-14
Service asks the rules	handler/LessonService.java : 16-19
Start-up demo	handler/LessonDemoRunner.java : 20-43
Vendor status -> LessonStatus	dto/VendorLessonPayload.java : 17-27

All paths are relative to `src/main/java/com/endev/tutoring/`.